Android WebView APK — any web app, packaged

Turn any HTTPS web app into an installable Android app with .NET — no Java, no Android Studio · for Luís

> The recipe: a **.NET for Android** project (~180 lines of C#) whose only screen is a full-screen **WebView** — Chrome's engine, embeddable in any Android app — pinned to your web app's URL. The real application keeps running on the server; the phone just renders it. Deploy to the server and every phone updates **instantly** — the APK is only rebuilt when the wrapper itself changes (URL, icon, native behavior). Worked example: the *Ramil Técnico* app at `SRamil\Mobile\RamilTecnico\`.

1. When this is the right tool

- The web app is already **mobile-friendly** (responsive layout, touch-sized targets).
- You want a launcher icon, camera access, and a "real app" feel — without maintaining a native codebase.
- Users are a known group who can **sideload** (no Play Store review, no store account).
- Connectivity is assumed — a WebView wrapper has no offline mode (Blazor Server especially).

2. Prerequisites (one-time, per machine)

1. The .NET for Android workload — Visual Studio's mobile option installs it; otherwise:

```
dotnet workload install android
```

2. An Android SDK. Visual Studio puts one at `C:\Program Files (x86)\Android\android-sdk`. If missing, one command downloads SDK + JDK into a folder you choose:

```
dotnet build -t:InstallAndroidDependencies -f net10.0-android `
  -p:AndroidSdkDirectory="%LOCALAPPDATA%\Android\sdk" -p:AcceptAndroidSDKLicenses=True
```

That's all. No Android Studio, no Google account — the output is an .apk file installed directly.

3. The project — four files, copy and adapt

Create a folder (outside your web project's build globs — see the warning in §4) with these files. The three placeholders to change: `YOUR_HOST`, `com.yourcompany.yourapp`, and the app name.

3.1 MyApp.csproj

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <TargetFramework>net10.0-android</TargetFramework>
    <SupportedOSPlatformVersion>24.0</SupportedOSPlatformVersion>
    <OutputType>Exe</OutputType>
    <Nullable>enable</Nullable>
    <ImplicitUsings>enable</ImplicitUsings>
    <ApplicationId>com.yourcompany.yourapp</ApplicationId>
    <ApplicationVersion>1</ApplicationVersion>
    <ApplicationDisplayVersion>1.0</ApplicationDisplayVersion>
    <!-- Sideload distribution: a plain APK, not an AAB. -->
    <AndroidPackageFormat>apk</AndroidPackageFormat>
  </PropertyGroup>
  <!-- Release signing lives in a gitignored file (see §8). -->
  <Import Project="signing.props" Condition="Exists('signing.props')" />
</Project>
```

3.2 Properties\AndroidManifest.xml

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android">
  <uses-permission android:name="android.permission.INTERNET" />
  <!-- Only if the page uses the camera (QR scanning etc.) -->
  <uses-permission android:name="android.permission.CAMERA" />
  <uses-feature android:name="android.hardware.camera" android:required="false" />
  <application android:label="My App" android:icon="@mipmap/ic_launcher"
               android:usesCleartextTraffic="false" />
</manifest>
```

`usesCleartextTraffic="true"` only for temporary HTTP testing — and remember the camera never works over HTTP (§9).

3.3 Resources\values\styles.xml

```
<?xml version="1.0" encoding="utf-8"?>
<resources>
  <!-- No action bar: the web app draws its own header. -->
  <style name="AppTheme" parent="@android:style/Theme.Material.Light.NoActionBar">
    <item name="android:statusBarColor">#1F7A3D</item>  <!-- your brand color -->
    <item name="android:windowBackground">@android:color/white</item>
  </style>
</resources>
```

3.4 MainActivity.cs — the whole app

This is the complete file. Each region solves one thing a plain browser tab can't; delete what you don't need (e.g. the camera bridge if the page never uses it).

```
using Android;
using Android.App;
using Android.Content;
using Android.Content.PM;
using Android.Graphics;
using Android.OS;
using Android.Views;
using Android.Webkit;
using Android.Widget;

namespace MyApp;

[Activity(Label = "My App", MainLauncher = true, Exported = true,
    Theme = "@style/AppTheme",
    WindowSoftInputMode = SoftInput.AdjustResize,
    ConfigurationChanges = ConfigChanges.Orientation | ConfigChanges.ScreenSize | ConfigChanges.KeyboardHidden)]
public class MainActivity : Activity
{
    // THE deployment knob. HTTPS is mandatory for camera/mic (getUserMedia).
    internal const string BaseHost = "YOUR_HOST";              // e.g. app.ramil.pt
    private const string  StartUrl = $"https://{BaseHost}/";   // + entry path if any

    private const int CameraPermissionRequestCode = 1001;
    private WebView? _web;
    private PermissionRequest? _pendingWebPermission;

    protected override void OnCreate(Bundle? savedInstanceState)
    {
        base.OnCreate(savedInstanceState);
        _web = new WebView(this);
        _web.SetBackgroundColor(Color.White);

        // Android 15+ draws apps EDGE-TO-EDGE (behind status/navigation bars) — pad for
        // them ourselves. Top strip = brand color (merges with the web app's header),
        // bottom = white; the keyboard inset keeps inputs visible.
        var root = new FrameLayout(this);
        root.SetBackgroundColor(Color.ParseColor("#1F7A3D"));  // your brand color
        root.AddView(_web);
        SetContentView(root);

        if (Build.VERSION.SdkInt >= BuildVersionCodes.R)
        {
            root.SetOnApplyWindowInsetsListener(new SystemBarInsetsListener(_web));
            Window?.InsetsController?.SetSystemBarsAppearance(
                (int)WindowInsetsControllerAppearance.LightNavigationBars,
                (int)WindowInsetsControllerAppearance.LightNavigationBars);
        }

        var s = _web.Settings;
        s.JavaScriptEnabled = true;
        s.DomStorageEnabled = true;
        s.MediaPlaybackRequiresUserGesture = false;
        s.MixedContentMode = MixedContentHandling.NeverAllow;

        CookieManager.Instance!.SetAcceptCookie(true);   // login survives restarts
        CookieManager.Instance.SetAcceptThirdPartyCookies(_web, false);

        _web.SetWebViewClient(new AppWebViewClient(this));
        _web.SetWebChromeClient(new AppChromeClient(this));

        if (savedInstanceState is null) _web.LoadUrl(StartUrl);
        else _web.RestoreState(savedInstanceState);
    }

    protected override void OnSaveInstanceState(Bundle outState)
    { base.OnSaveInstanceState(outState); _web?.SaveState(outState); }

    protected override void OnPause()
    {
```

```
        base.OnPause();
        // Cookie changes are persisted LAZILY — an app killed right after a LOGOUT would
        // lose the cookie deletion and resurrect the session on relaunch. Flush on pause.
        CookieManager.Instance?.Flush();
    }

    public override void OnBackPressed()
    { if (_web?.CanGoBack() == true) _web.GoBack(); else base.OnBackPressed(); }

    // ── Camera bridge: page asks WebView → WebView asks app → app asks Android ──
    internal void OnWebPermissionRequest(PermissionRequest request)
    {
        var wantsCamera = request.GetResources()?.Contains(PermissionRequest.ResourceVideoCapture) == true;
        var origin = request.Origin?.Host;
        if (!wantsCamera || !string.Equals(origin, BaseHost, StringComparison.OrdinalIgnoreCase))
        { request.Deny(); return; }

        if (CheckSelfPermission(Manifest.Permission.Camera) == Permission.Granted)
        { request.Grant(new[] { PermissionRequest.ResourceVideoCapture }); return; }

        _pendingWebPermission = request;
        RequestPermissions(new[] { Manifest.Permission.Camera }, CameraPermissionRequestCode);
    }

    public override void OnRequestPermissionsResult(int requestCode, string[] permissions, Permission[] grantResults)
    {
        base.OnRequestPermissionsResult(requestCode, permissions, grantResults);
        if (requestCode != CameraPermissionRequestCode || _pendingWebPermission is null) return;
        if (grantResults.FirstOrDefault() == Permission.Granted)
            _pendingWebPermission.Grant(new[] { PermissionRequest.ResourceVideoCapture });
        else _pendingWebPermission.Deny();
        _pendingWebPermission = null;
    }
}

// Edge-to-edge insets: status bar padded on the (brand-colored) container, navigation
// bar or keyboard — whichever is taller — padded on the (white) WebView.
internal sealed class SystemBarInsetsListener : Java.Lang.Object, View.IOnApplyWindowInsetsListener
{
    private readonly WebView _web;
    public SystemBarInsetsListener(WebView web) => _web = web;
    public WindowInsets OnApplyWindowInsets(View v, WindowInsets insets)
    {
        var bars = insets.GetInsets(WindowInsets.Type.SystemBars());
        var ime = insets.GetInsets(WindowInsets.Type.Ime());
        v.SetPadding(bars.Left, bars.Top, bars.Right, 0);
        _web.SetPadding(0, 0, 0, Math.Max(bars.Bottom, ime.Bottom));
        return insets;
    }
}

// Relays the page's permission requests (camera) onto the UI thread.
internal sealed class AppChromeClient : WebChromeClient
{
    private readonly MainActivity _activity;
    public AppChromeClient(MainActivity activity) => _activity = activity;
    public override void OnPermissionRequest(PermissionRequest? request)
    { if (request is null) return; _activity.RunOnUiThread(() => _activity.OnWebPermissionRequest(request)); }
}

// Our host stays in-app; everything else (mailto:, tel:, other sites) → the system.
internal sealed class AppWebViewClient : WebViewClient
{
    private readonly MainActivity _activity;
    public AppWebViewClient(MainActivity activity) => _activity = activity;
    public override bool ShouldOverrideUrlLoading(WebView? view, IWebResourceRequest? request)
    {
        var uri = request?.Url;
        if (uri is null) return false;
        var isHttp = uri.Scheme is "http" or "https";
```

```
        if (isHttp && string.Equals(uri.Host, MainActivity.BaseHost, StringComparison.OrdinalIgnoreCase))
            return false;
        try { _activity.StartActivity(new Intent(Intent.ActionView, uri)); }
        catch (ActivityNotFoundException) { }
        return true;
    }
}
```

> **If the folder lives inside a web project** (like `SRamil\Mobile\`), exclude it from that project's default globs or the site build breaks with duplicate-attribute errors: `<Compile Remove="Mobile\**" />` (+ Content/EmbeddedResource/None) in the web .csproj.

4. Build the APK

```
cd <project folder>
dotnet publish -c Release -p:AndroidSdkDirectory="C:\Program Files (x86)\Android\android-sdk"
# output:
bin\Release\net10.0-android\<ApplicationId>-Signed.apk   (~7 MB)
```

The **-Signed.apk** is the distributable. First build takes minutes; subsequent ones ~20 seconds.

5. Install on a phone (sideload)

1. Send the .apk (WhatsApp, e-mail, Drive, USB).
2. Tap it; allow "install unknown apps" for that source (asked once).
3. On the first camera use, Android prompts for the permission — accept.

6. Icon

1. A square source image (512×512, white or full-bleed — launchers crop to a circle).
2. Resize into `Resources\mipmap-*\ic_launcher.png`: 48 (mdpi), 72 (hdpi), 96 (xhdpi), 144 (xxhdpi), 192 px (xxxhdpi).
3. Rebuild. Some launchers only refresh the icon after uninstall/reinstall.

7. Signing — before real distribution

Unconfigured builds sign with a machine-local **debug** key. Rule: **updates only install over the old version if signed with the SAME key** — lose it and everyone reinstalls from scratch. Create a release keystore once, keep it safe and out of git, reference it in `signing.props` (auto-imported):

```
<!-- signing.props — NEVER enters git -->
<Project><PropertyGroup>
  <AndroidKeyStore>true</AndroidKeyStore>
  <AndroidSigningKeyStore>myapp.keystore</AndroidSigningKeyStore>
  <AndroidSigningKeyAlias>myapp</AndroidSigningKeyAlias>
  <AndroidSigningKeyPass>A_PASSWORD</AndroidSigningKeyPass>
  <AndroidSigningStorePass>A_PASSWORD</AndroidSigningStorePass>
</PropertyGroup></Project>
# create the keystore (keytool ships with any JDK):
keytool -genkeypair -v -keystore myapp.keystore -alias myapp -keyalg RSA -keysize 2048 -validity 10000
```

8. The worked example — Ramil Técnico

Knob	Value
Project	`SRamil\Mobile\RamilTecnico\`
ApplicationId	`pt.agileincloud.ramil.tecnico`
BaseHost / StartUrl	`app.ramil.pt` → `https://app.ramil.pt/tecnico`
Brand color (status bar)	`#1F7A3D`
Camera	Yes — the QR scanner uses getUserMedia

9. Troubleshooting

Symptom	Cause & fix
Camera won't open / black screen	The URL is HTTP. getUserMedia only works on HTTPS — a Chrome/Android security rule, not a bug.
Logged out, reopened the app, logged in again	Cookie deletions are persisted lazily; without `CookieManager.Flush()` in `OnPause`, killing the app resurrects the session cookie. The template handles it.
Content under the clock/battery bar	Android 15+ edge-to-edge; the insets listener in the template pads for the system bars.
Keyboard covers the input	The IME inset (template) + `WindowSoftInputMode.AdjustResize` handle it.
Blank page / connection error	The phone can't reach the server — test the URL in the phone's own Chrome first.
Update won't install	Different signing key (§7): uninstall first, or rebuild with the right keystore.
Web app changed — new APK?	**No.** Only wrapper changes (URL, icon, native behavior) need a rebuild.